

WEBRTC

The REALTIME GLUE between AGENTS and the WORLD

A Talk by

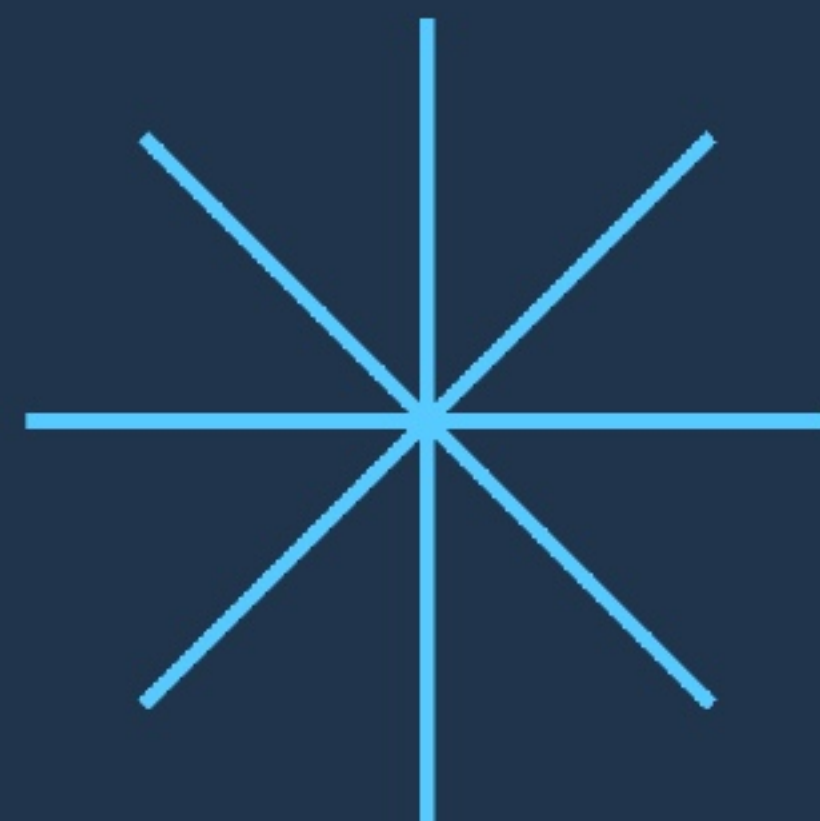
ETIENNE CARON



ETIENNE CARON
from Senior to Staff Developer



{ 2025 }



ETIENNE CARON

Android GDE

AMA
*STRUCK

INNOVATION LAB



ETIENNE CARON

Technical Founder

TALK OVERVIEW

WHAT ARE WE LOOKING AT TODAY?

**WEBRTC AS
GLUE? WHY?**

WEBRTC BASICS

HOW TO SELF-HOST

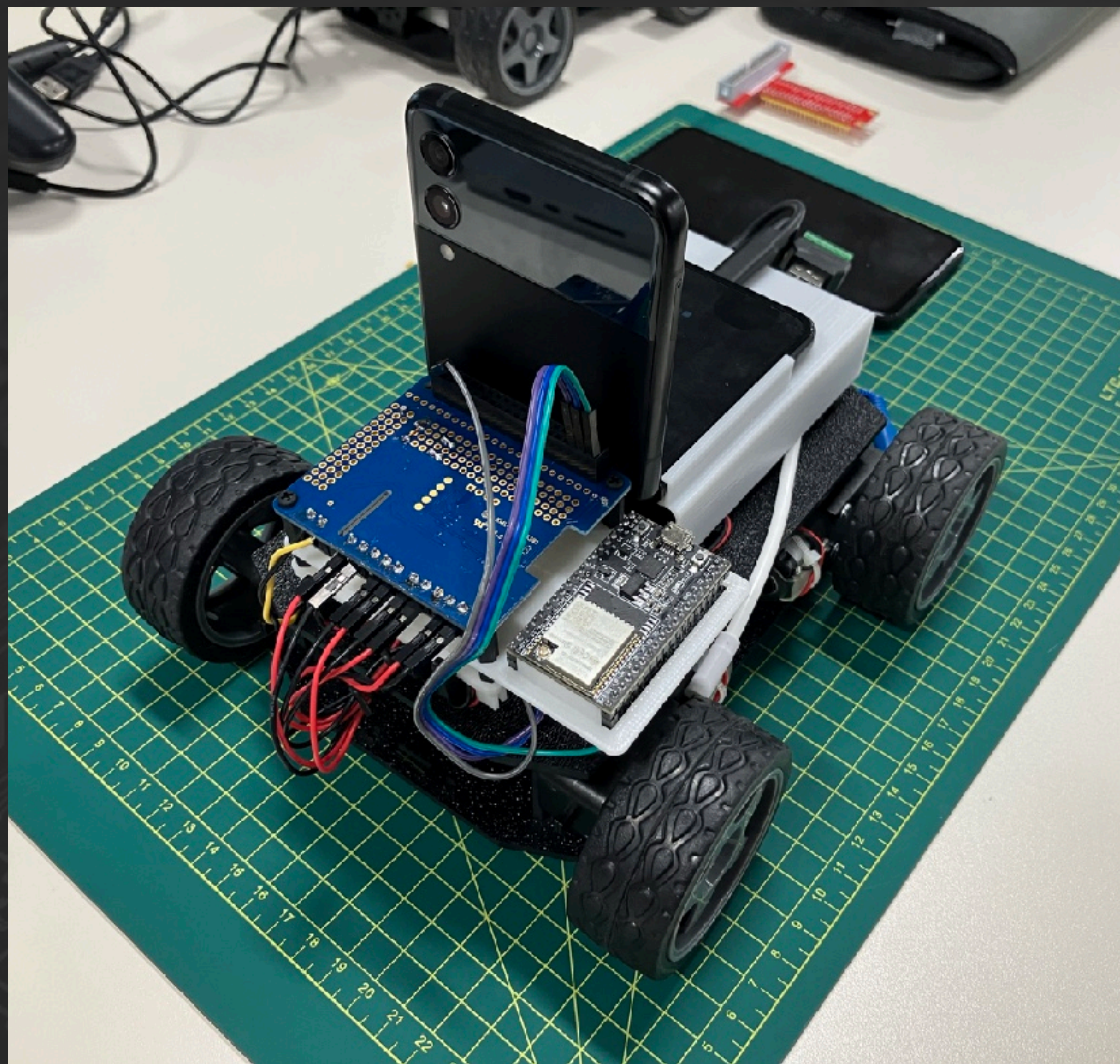
**AGENTS & REALTIME
VIDEO PROCESSING**

WEBRTC AS GLUE? WHY?

Course Bell

XANA
STRUCK





Preuve de concept: μ C, servos, serial over USB contrôlé via mobile



Entrainement et analyse via Yolo v8 + Digital Twin



Système de positionnement en temps réel

Shopping re-imagined





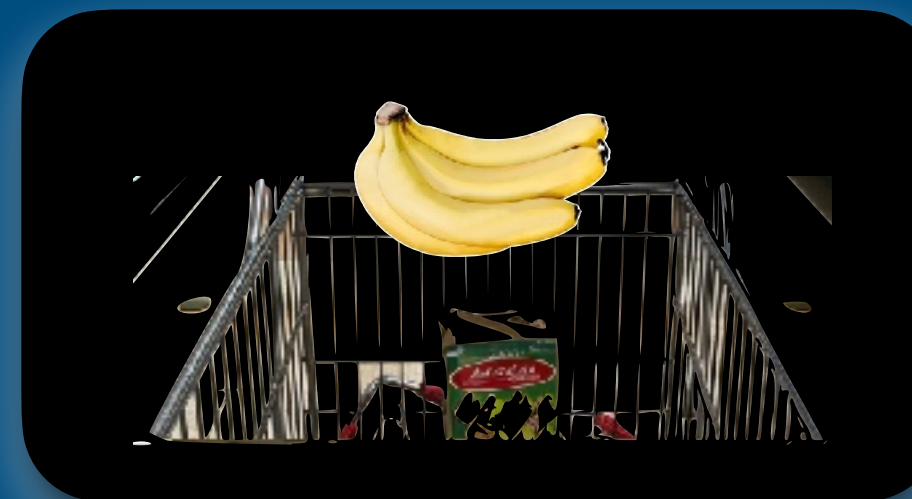
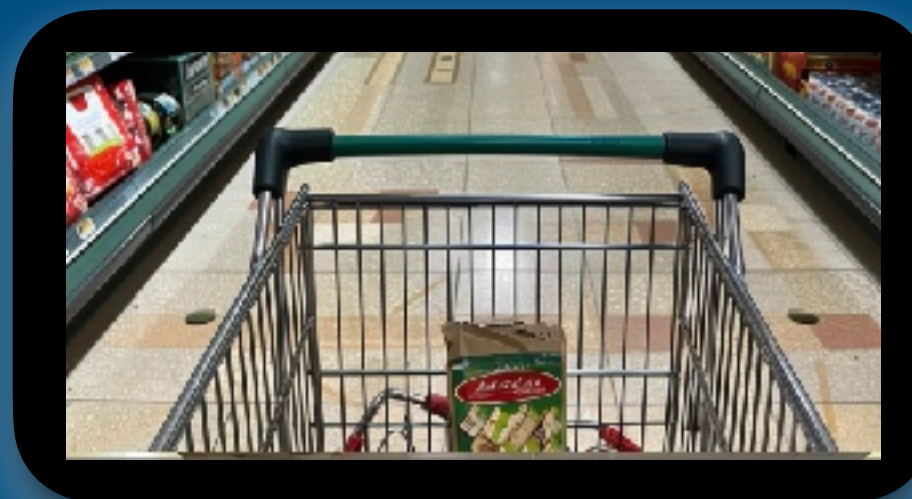
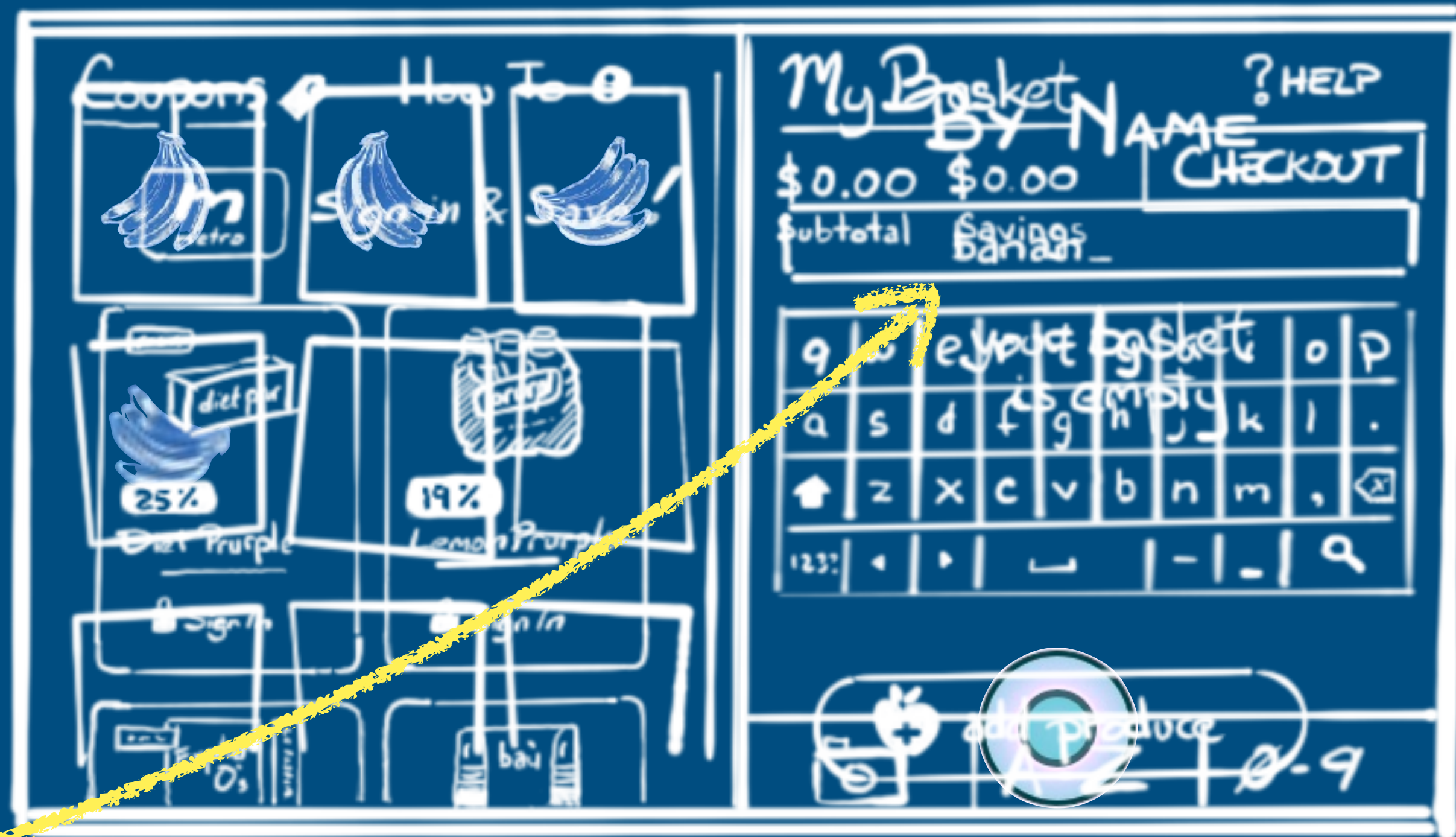
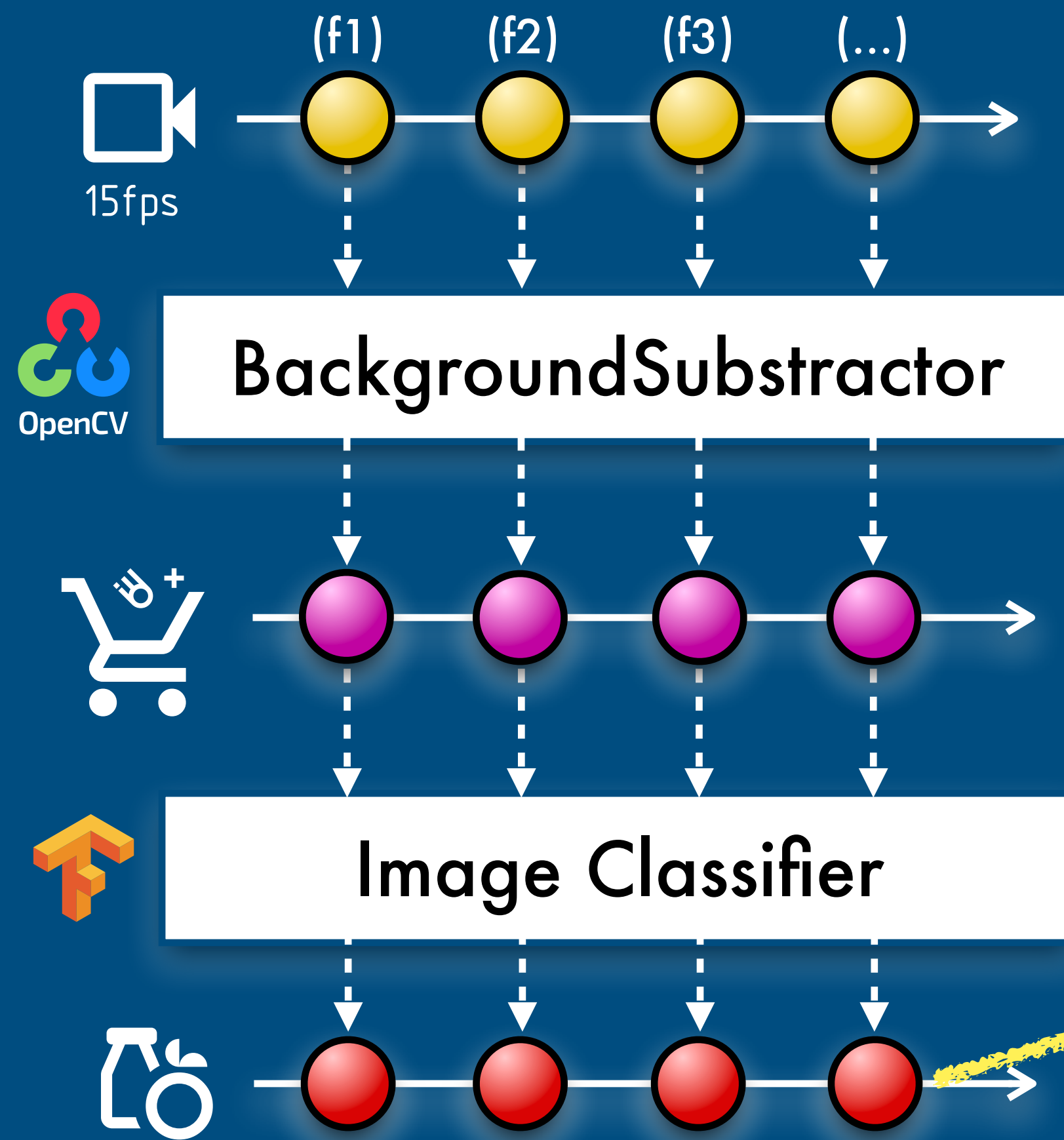
◆ AI Overview

The cost of a Caper cart is not publicly disclosed, but estimates suggest they are expensive for retailers, likely in the range of **\$5,000 to \$10,000 per cart**. This is a significant capital investment compared to a standard cart, which costs around \$100. The total cost to a retailer for implementing the technology could be in the **hundreds of thousands of dollars**, depending on the number of carts deployed. [🔗](#)



Cost breakdown

- **Per-cart cost:** Estimated to be between \$5,000 and \$10,000 each.
- **Total implementation cost:** Retailers deploying a large number of carts could face



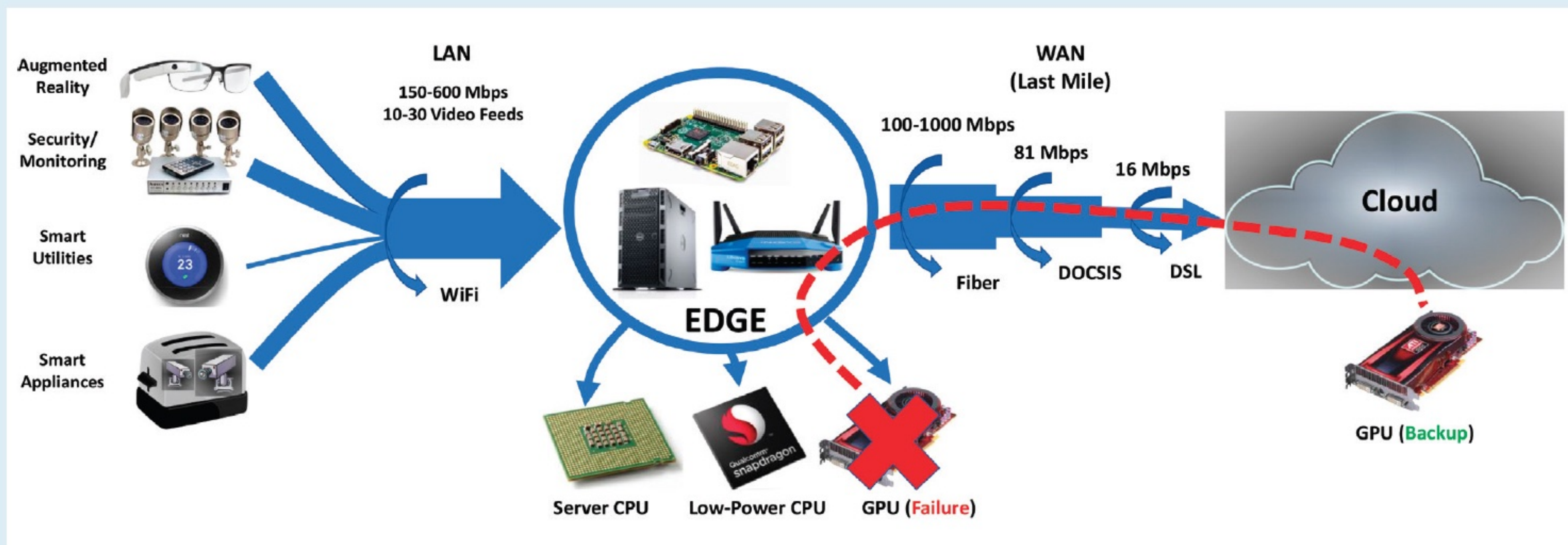
AINED

ast

Hand-drawn customer work.

AUTHOR: @kanawish
DATE: 24/04/20

The high bandwidth of WiFi allows edge computers such as ARM-based Raspberry Pi's, routers, and traditional tower servers to process the data. When there is a failure, the data can be re-routed to the cloud, but this re-routing is limited by the capacity of the WAN system.



<https://cacm.acm.org/research/dependability-in-edge-computing>

QuickTime PlayerFileEditViewWindowHelp

SM-F711W

11:34 39%

WebRTC KMP

Gamepad: DUALSHOCK 4 Wireless Controller (STANDARD GAMEPAD Vendor: 054c Product: 05)

L2R2
L1R1

L(0.004,0.004)R(0.004,0.004)

Room ID: 01iKUS980ehztqqTtT2Y

Switch camera

Hangup

Photo Booth

Click Photo Booth to resume

bytesReceived

[bytesReceived_in_bits/s]

totalRoundTripTime

[totalRoundTripTime/responsesReceived]

currentRoundTripTime

availableOutgoingBitrate

JS Client

localho...

Update

WebRTC KMP Sample

ID: 01iKUS980chztqqTtT2Y

GP Index 0

CONNECT

Signed In: ctienne.caron@gmail.comSIGN OUT

OPEN CAMERA AND MICROPHONE

CREATE ROOM

JOIN ROOM

HANGUP



<https://reutilisons.com/>



Parlez à un expert

— Réemploi, réutilisation, revalorisation, On s'occupe de tout.

Votre **meuble** a de la **valeur**.
Donnez-lui une **seconde vie**.



Mobilier excédentaire

Centre-de-tri virtuel

Disposition écoresponsable

Solution zéro déchet

Donnez un nouveau souffle à vos déchets, en soutenant les communautés locales et en adoptant une gestion écoresponsable de votre inventaire de mobilier de bureau.



870+

Partenaires

1,571,895+

Livres détournées de sites d'enfouissement.

15,139+

Items remis en don.

760,981\$

Valeur monétaire remis en don.











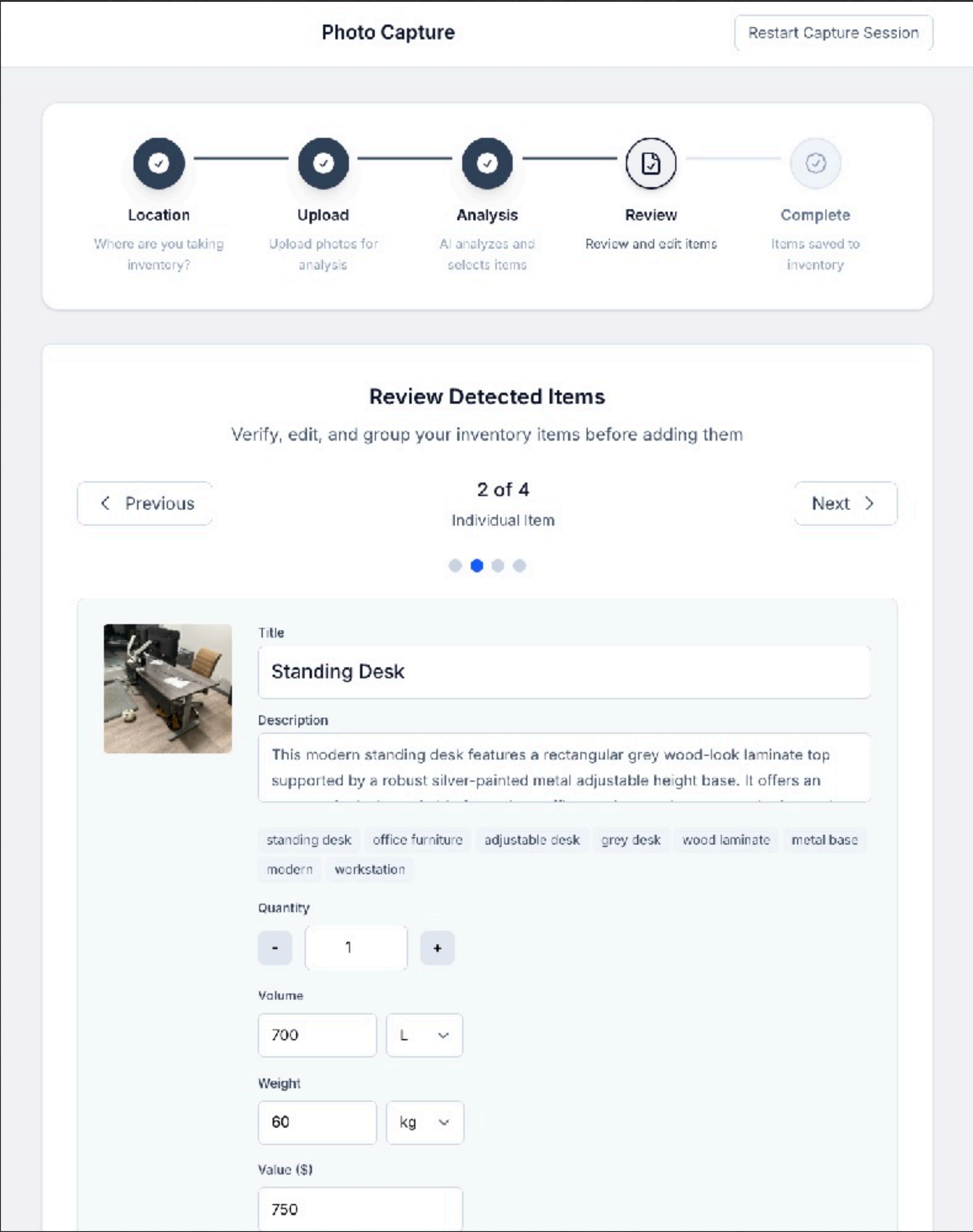
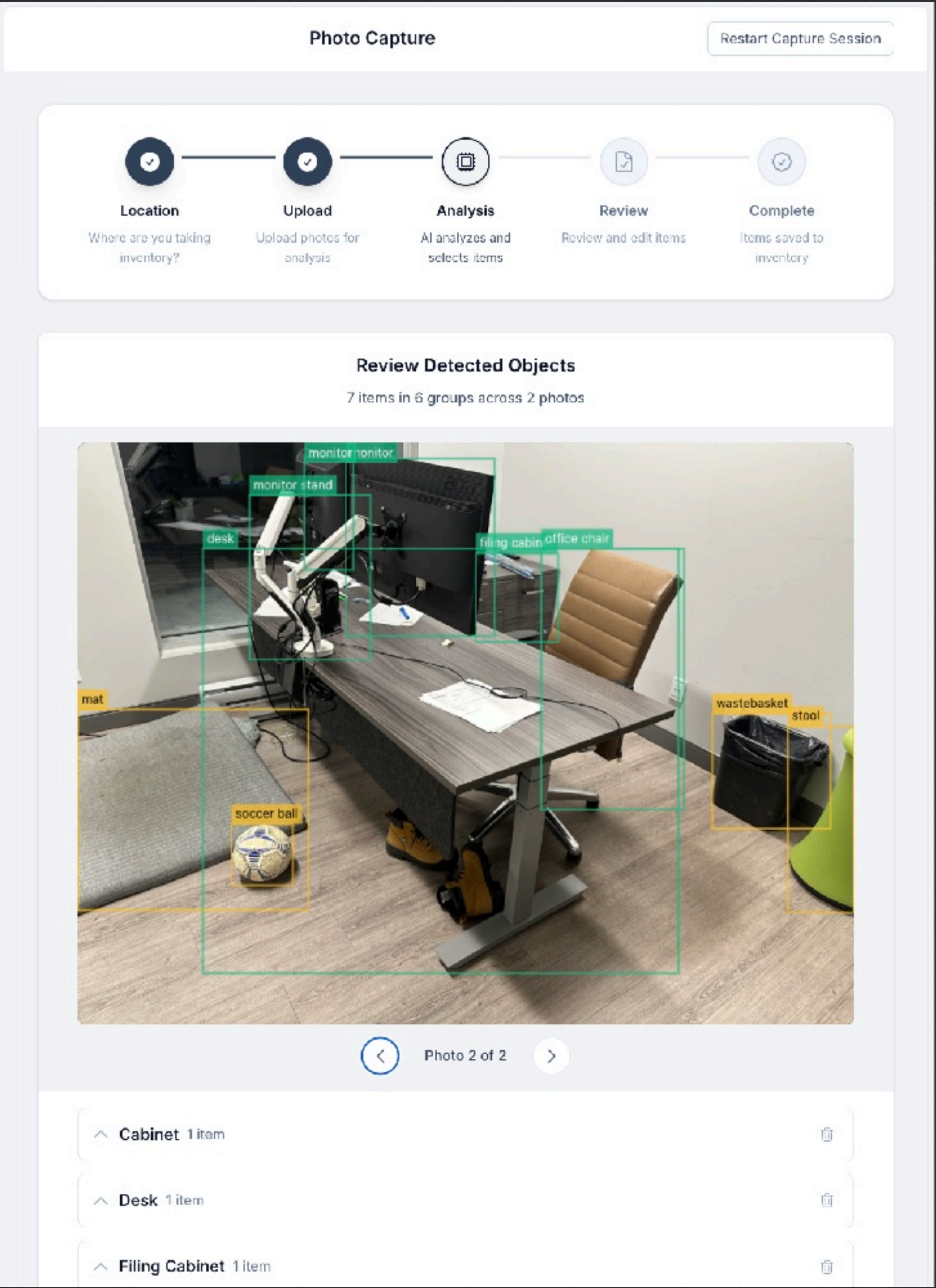


OFFICE INVENTORY TRIGGERS

WHEN TOMORROW BECOMES TODAY

- * Moves, Renovations, Storage
- * Sustainability or Reuse Audits
- * Insurance policy Audits
- * Theft, Fire, Flood Damage Audits
- * IT Infrastructure Upgrades





WEBRTC BASICS

WEBRTC CONNECTIVITY

IN ORDER OF PREFERENCE

- **ICE over UDP:** ideal connection type, used in majority of conditions
- **TURN with UDP (3478):** used when ICE/UDP is unreachable
- **ICE over TCP:** used when network disallows UDP (i.e. over VPN or corporate firewalls)
- **TURN with TLS:** used when firewall only allows outbound TLS connections

Wut?



ICE

*Interactive Connectivity
Establishment*



STUN

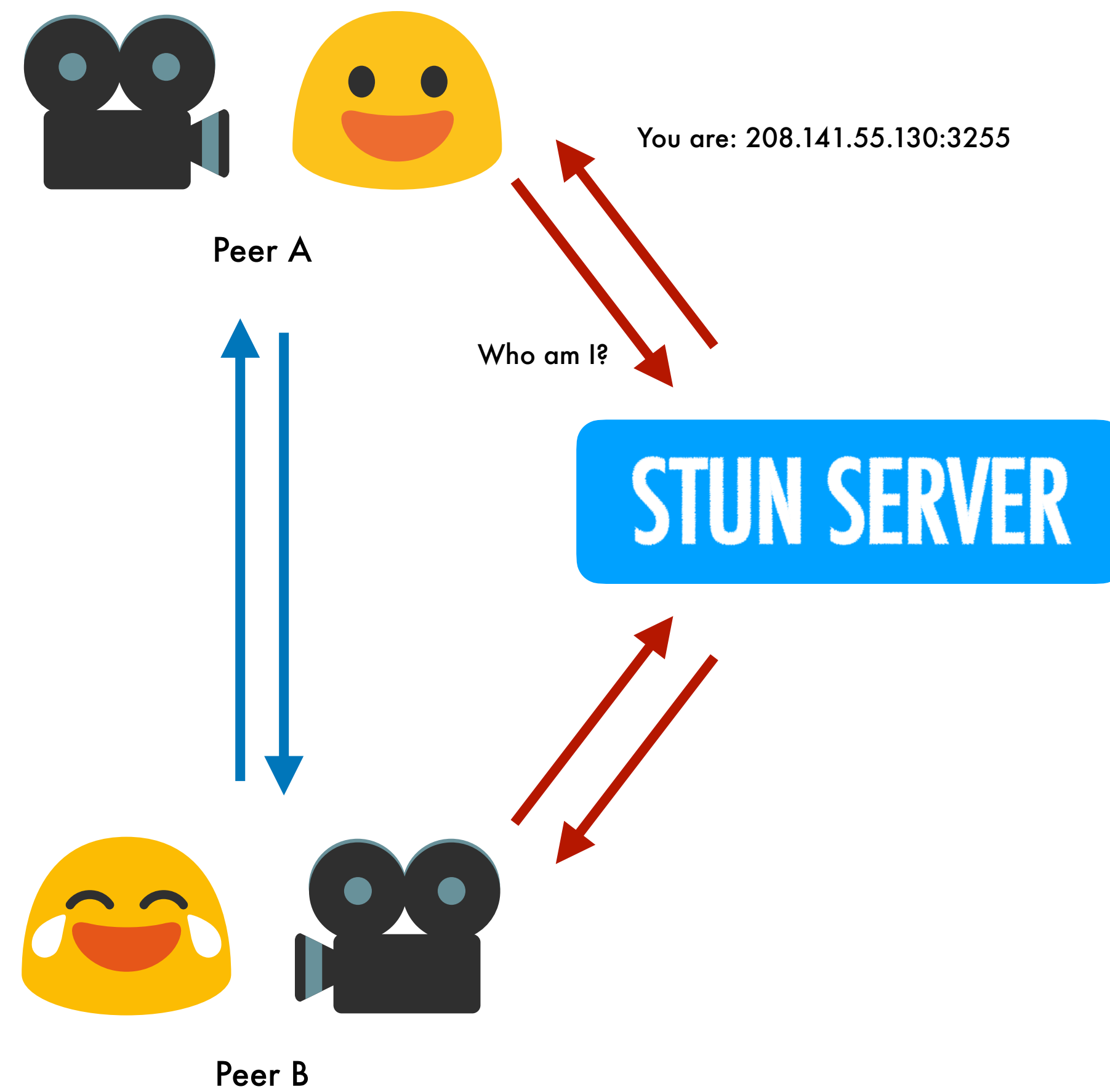
*Session Traversal Utilities for
NAT*

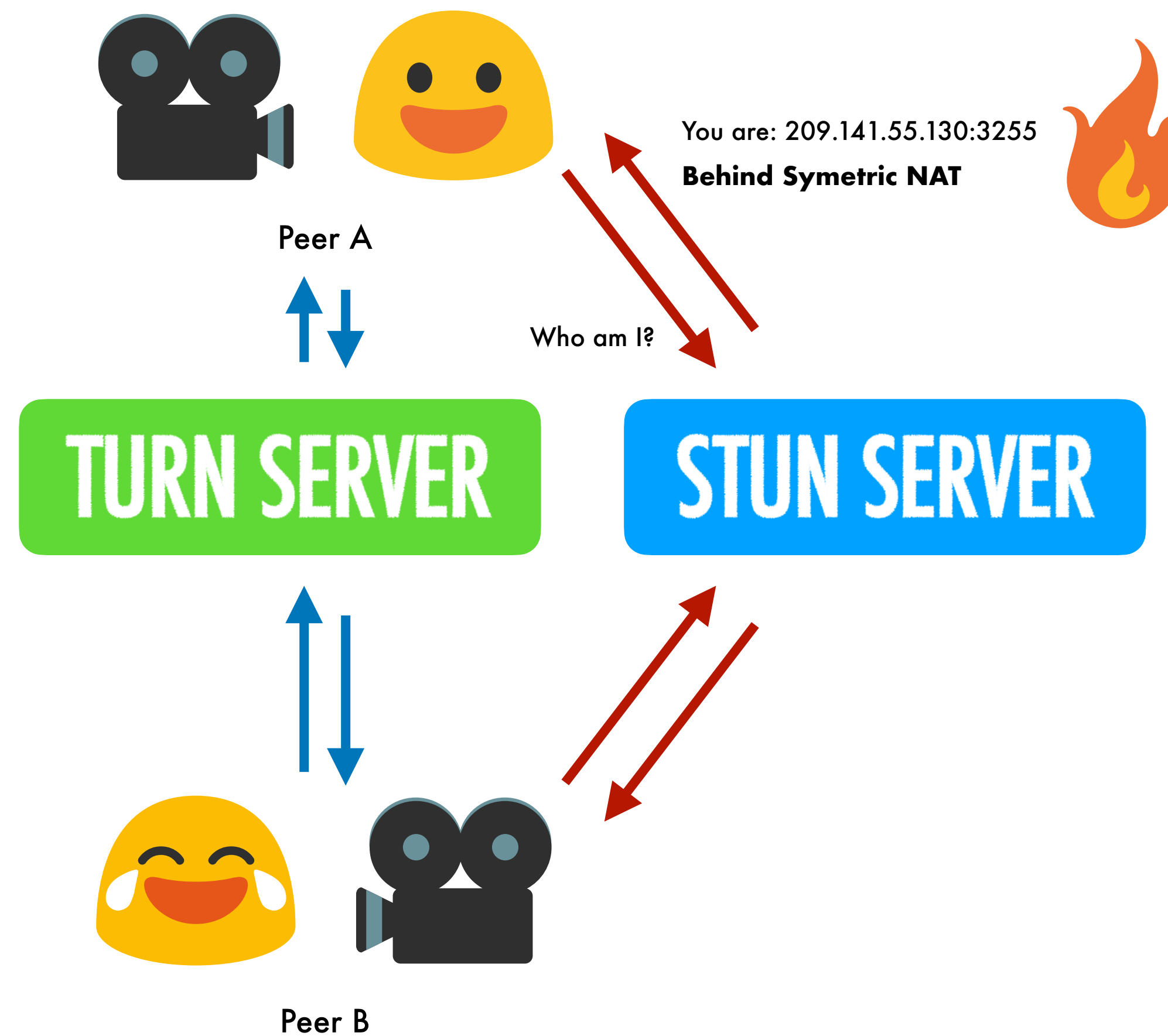


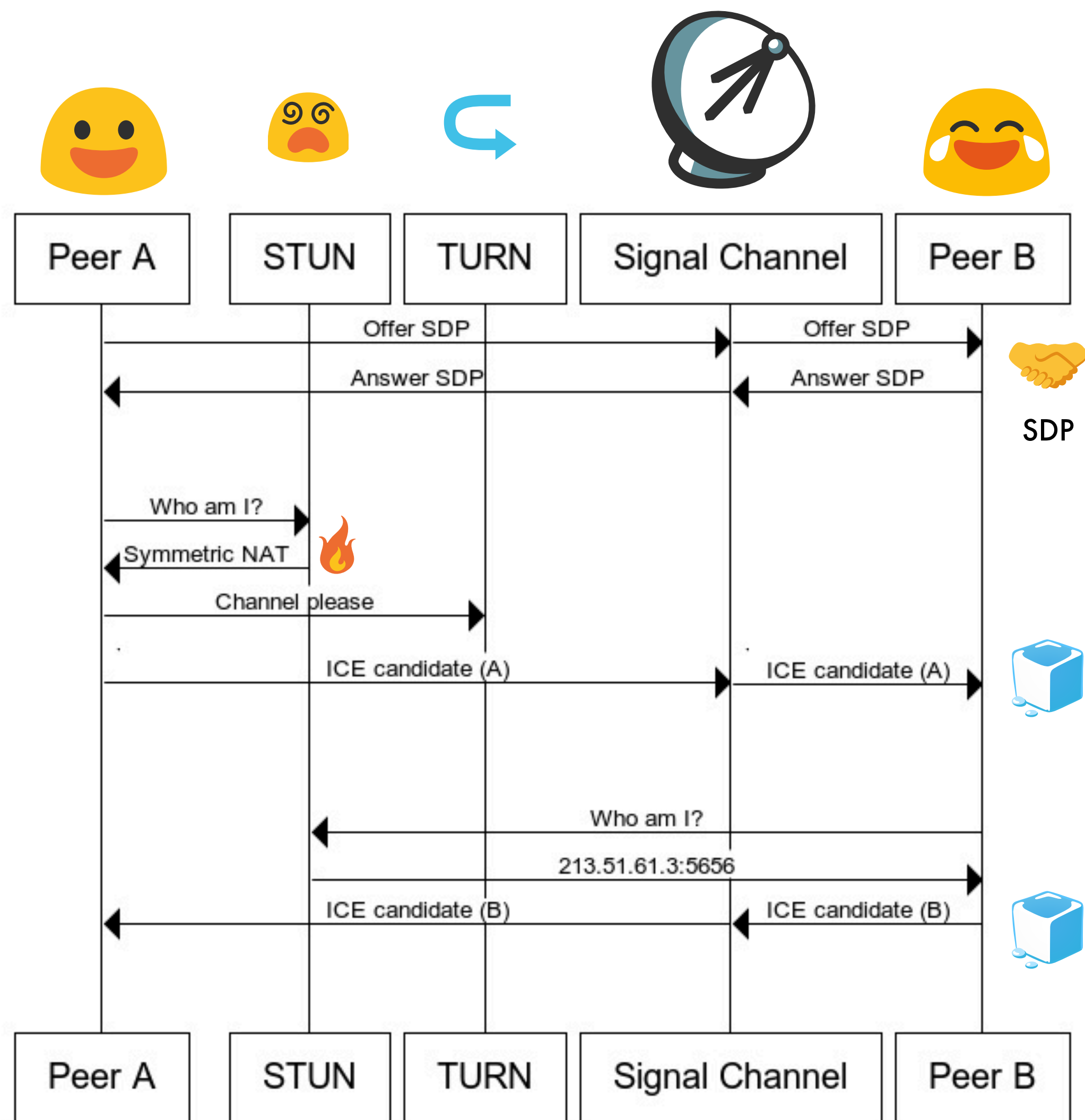
TURN

*Traversal Using Relays around
NAT*









https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Connectivity#the_entire_exchange_in_a_complicated_diagram

STUN

TURN

Signal Channel

Peer B

developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Activity #the entire exchange in a complicated diagram



Open source framework for voice and multimodal conversational AI.

Supported by the Pipecat community and the [Daily.co](#) engineering team.

Try Pipecat

OpenAI Realtime

Connect







.....

Transcript appears here

[Browse community events](#) ➤

[Github](#) [Docs](#) [Discord](#)

<https://www.pipecat.ai/>



Pipecat

pypi v0.0.103 tests passing codecov 29% Documentation chat 570 online Ask DeepWiki

Pipecat: Real-Time Voice & Multimodal AI Agents

Pipecat is an open-source Python framework for building real-time voice and multimodal conversational agents. Orchestrate audio and video, AI services, different transports, and conversation pipelines effortlessly—so you can focus on what makes your agent unique.

Want to dive right in? Try the [quickstart](#).

What You Can Build

- **Voice Assistants** – natural, streaming conversations with AI
- **AI Companions** – coaches, meeting assistants, characters
- **Multimodal Interfaces** – voice, video, images, and more
- **Interactive Storytelling** – creative tools with generative media
- **Business Agents** – customer intake, support bots, guided flows
- **Complex Dialog Systems** – design logic with structured conversations

Why Pipecat?

- **Voice-first:** Integrates speech recognition, text-to-speech, and conversation handling
- **Pluggable:** Supports many AI services and tools
- **Composable Pipelines:** Build complex behavior from modular components
- **Real-Time:** Ultra-low latency interaction with different transports (e.g. WebSockets or WebRTC)

Pipecat Ecosystem

Client SDKs

Building client applications? You can connect to Pipecat from any platform using our official SDKs:

[JavaScript](#) | [React](#) | [React Native](#) | [Swift](#) | [Kotlin](#) | [C++](#) | [ESP32](#)

<https://github.com/pipecat-ai/pipecat>

Why Pipecat?

- **Voice-first:** Integrates speech recognition, text-to-speech, and conversation handling
- **Pluggable:** Supports many AI services and tools
- **Composable Pipelines:** Build complex behavior from modular components
- **Real-Time:** Ultra-low latency interaction with different transports (e.g. WebSockets or WebRTC)

Pipecat Ecosystem

Client SDKs

Building client applications? You can connect to Pipecat from any platform using our official SDKs:

[JavaScript](#) | [React](#) | [React Native](#) | [Swift](#) | [Kotlin](#) | [C++](#) | [ESP32](#)

Structured conversations

<https://github.com/pipecat-ai/pipecat>

GET STARTED

- Overview
- About LiveKit
- Docs MCP server
- DeepLearning course
- Community resources

UNDERSTANDING LIVEKIT

- Overview
- LiveKit CLI >
- LiveKit Cloud
- Connecting to LiveKit
- Rooms, participants, & tracks >
- Building AI agents

REFERENCE

- Recipes
- Room service API

About LiveKit

 Copy page

⌵

An overview of the LiveKit ecosystem.

What is LiveKit?

LiveKit is an open source framework and cloud platform for building voice, video, and physical AI agents. It provides the tools you need to build agents that interact with users in realtime over audio, video, and data streams. Agents run on the LiveKit server, which supplies the low-latency infrastructure — including transport, routing, synchronization, and session management — built on a production-grade WebRTC stack. This architecture enables reliable and performant agent workloads.

About WebRTC

The internet's core protocols weren't designed for realtime media. Hypertext Transfer Protocol (HTTP) is optimized for request-response communication, which is effective for the web's client-server model, but not for continuous audio and video streams. Historically, developers building realtime media applications had to work directly with the complexities of WebRTC.

WebRTC is a browser-native technology for transmitting audio and video in realtime. Unlike general-purpose transports such as websockets, WebRTC is optimized for media delivery, providing efficient codecs and automatically adapting to unreliable network conditions. Because all major browsers support WebRTC, it works consistently across platforms. LiveKit manages the operational and scaling challenges of WebRTC and extends its use to mobile applications, backend services, and telephony integrations.

Why use LiveKit?

LiveKit differentiates itself through several key advantages:

Build faster with high-level abstractions: Use the LiveKit Agents framework to quickly build production-ready AI agents with built-in support for speech processing, turn-taking, multimodal events, and LLM integration. When you need custom behavior, access lower-level WebRTC primitives for complete control.

Write once, deploy everywhere: Both human clients and AI agents use the same SDKs and APIs, so you can write agent logic once and deploy it across Web, iOS, Android, Flutter, Unity, and backend environments. Agents and clients interact seamlessly regardless of platform.

Focus on building, not infrastructure: LiveKit handles the operational complexity of WebRTC so developers can focus on building agents. Choose between fully managed LiveKit Cloud or self-hosted deployment — both offer identical APIs and core capabilities.

Connect to any system: Extend LiveKit with egress, ingress, telephony, and server APIs to build end-to-end workflows that span web, mobile, phone networks, and physical devices.

On this page

- What is LiveKit?
- About WebRTC
- Why use LiveKit?
- What can I build?
- How does LiveKit work?
- LiveKit server
- LiveKit Agents framework
- SDKs and clients
- Integration services
- How can I learn more?

Is this page helpful?





LiveKit Server

// -----

```
livekit-cli create-token
  --api-key <project_api_key> --api-secret <project_api_secret>
  --join --room my-first-room --identity bot
  --valid-for 24h

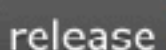
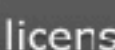
livekit-cli join-room
  --url <project_secure_websocket_address>
  --api-key <project_api_key> --api-secret <project_api_secret>
  --room my-first-room --identity bot
  --publish-demo
```

🔗 SERVICES

LiveKit: Real-time video, audio and data for developers

[LiveKit](#) is an open source project that provides scalable, multi-user conferencing based on WebRTC. It's designed to provide everything you need to build real-time video audio data capabilities in your applications.

LiveKit's server is written in Go, using the awesome [Pion WebRTC](#) implementation.

 Star
  Join
 10k
  6333 of us on Slack
  Follow @livekit
  release
 v1.8.0
  build
 passing
  license
 Apache-2.0

Features

- Scalable, distributed WebRTC SFU (Selective Forwarding Unit)
- Modern, full-featured client SDKs
- Built for production, supports JWT authentication
- Robust networking and connectivity, UDP/TCP/TURN
- Easy to deploy: single binary, Docker or Kubernetes

<https://github.com/livekit/livekit>

Features

- Scalable, distributed WebRTC SFU (Selective Forwarding Unit)
- Modern, full-featured client SDKs
- Built for production, supports JWT authentication
- Robust networking and connectivity, UDP/TCP/TURN
- Easy to deploy: single binary, Docker or Kubernetes
- Advanced features including:
 - [speaker detection](#)
 - [simulcast](#)
 - [end-to-end optimizations](#)
 - [selective subscription](#)
 - [moderation APIs](#)
 - end-to-end encryption
 - SVC codecs (VP9, AV1)
 - [webhooks](#)
 - [distributed and multi-region](#)

WEBRTC CONNECTIVITY

IN ORDER OF PREFERENCE



- **ICE over UDP:** ideal connection type, used in majority of conditions
- **TURN with UDP (3478):** used when ICE/UDP is unreachable
- **ICE over TCP:** used when network disallows UDP (i.e. over VPN or corporate firewalls)
- **TURN with TLS:** used when firewall only allows outbound TLS connections



Management Services

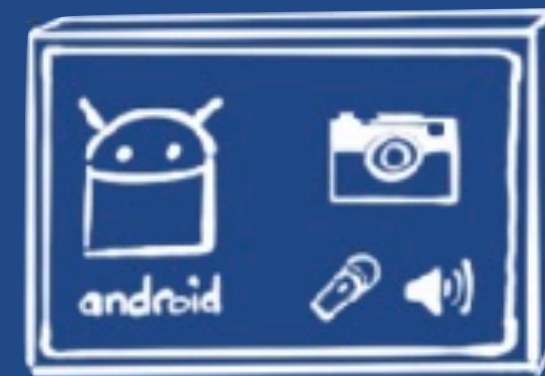
Authentication Service

Room Management Service

Client Terminal Service



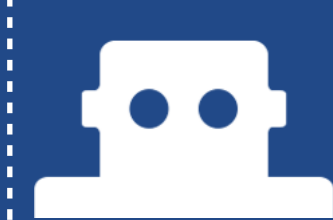
LiveKit Room



Client Terminals



Client



Agent



Support



Agent

Machine Learning Services

PyTorch TensorFlow

Object Detection

Object Classification

Text to Speech

Speech to Text

Generative AI

Natural Language Processing

Logging Services

Video Transcription

Media Stream Recording

Inference History

Audio Transcription



Realtime Media APIs

Image Processing

Encoding and Transcoding

Audio Processing

Filtering

Video Processing

Subtitles and Captioning



PLATFORM ARCHITECTURE DIAGRAM

Shows the components in more details.

AUTHOR: @kanawish
DATE: 24/04/20

HOW TO SELF-HOST

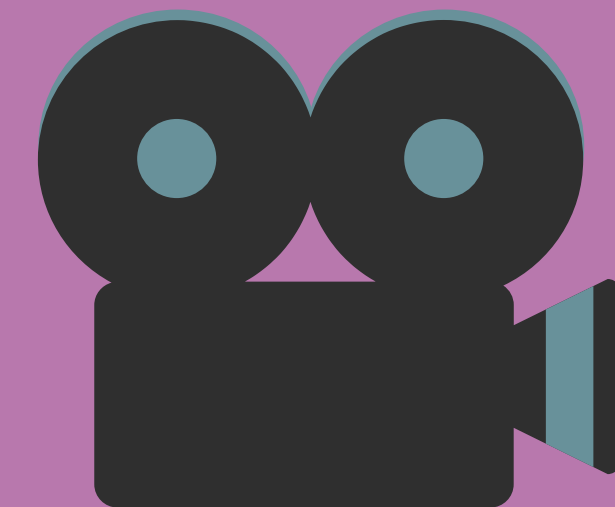
Room



Participant



Track



Running LiveKit locally

Copy page

This will get a LiveKit instance up and running, ready to receive audio and video streams from participants.

Install LiveKit Server

MACOS

LINUX

WINDOWS

```
brew update && brew install livekit
```

Start the server in dev mode

You can start LiveKit in development mode by running:

```
livekit-server --dev
```

This will start an instance using the following API key/secret pair:

```
API key: devkey
API secret: secret
```

To customize your setup for production, refer to our [deployment guides](#).



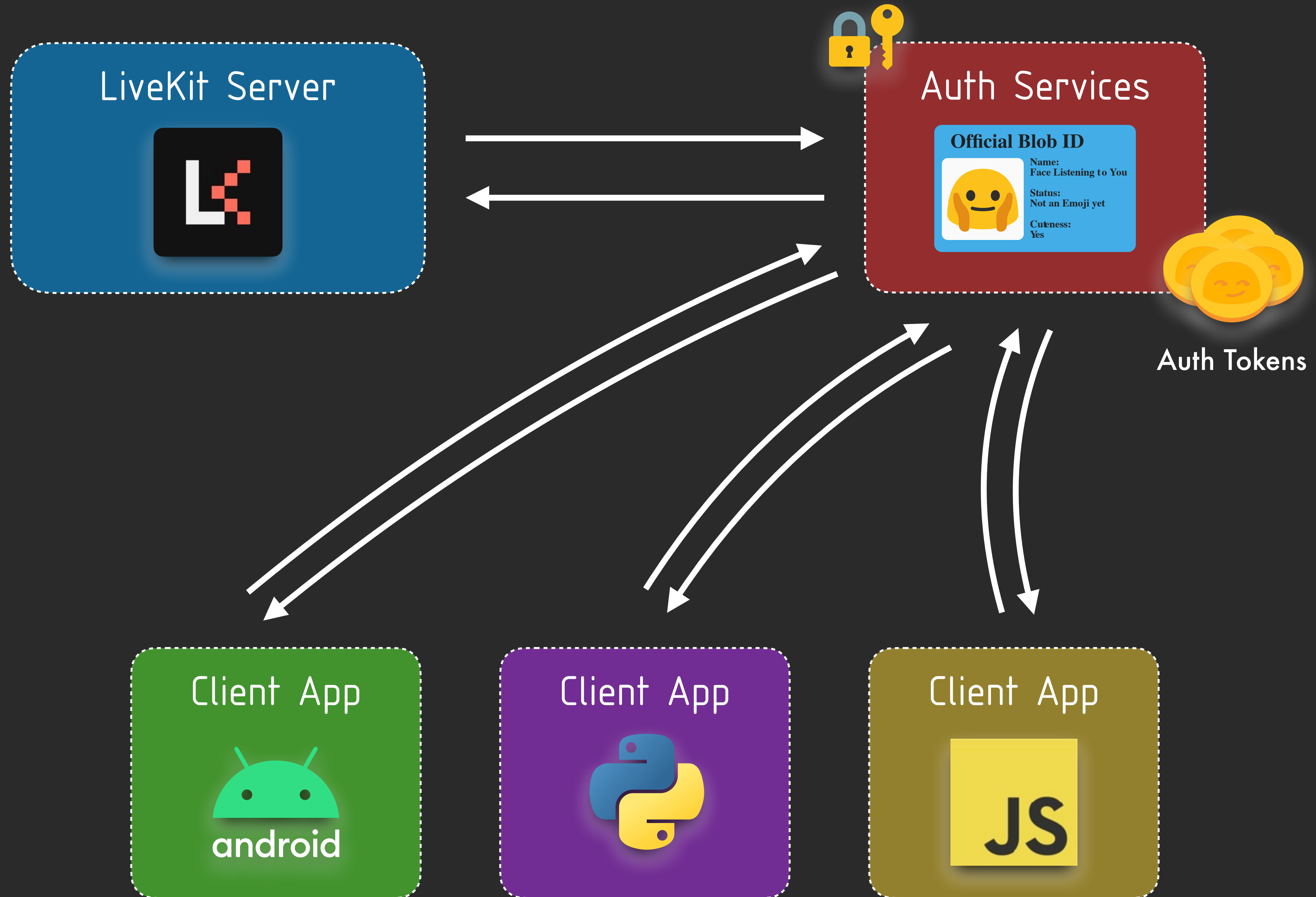
Tip

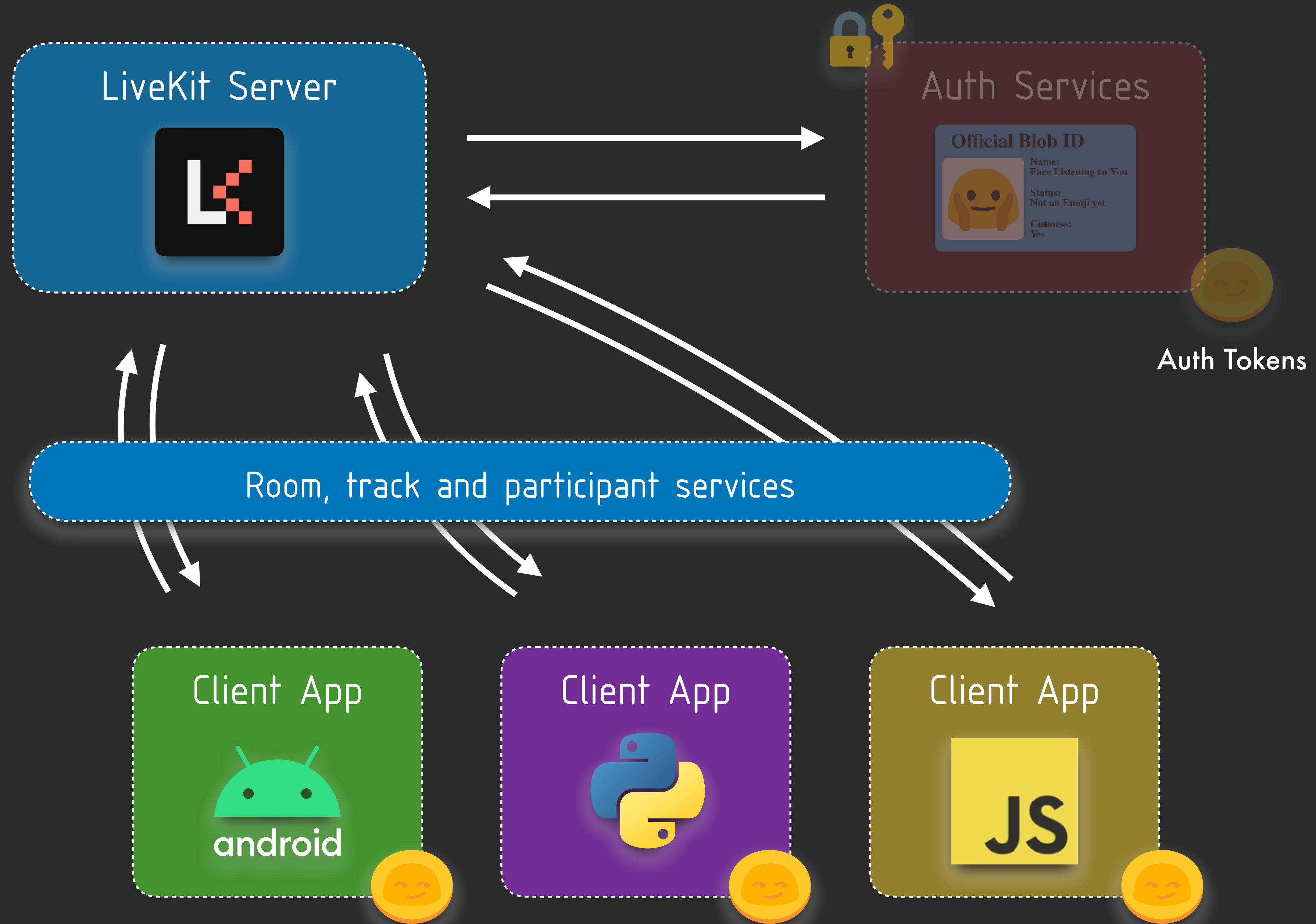
By default LiveKit's signal server binds to `127.0.0.1:7880`. If you'd like to access it from other devices on your network, pass in `--bind 0.0.0.0`

LiveKit Server



<https://docs.livekit.io/transport/self-hosting/local>







LiveKit Meet

[Try the demo](#) • [LiveKit Components](#) • [LiveKit Docs](#) • [LiveKit Cloud](#) • [Blog](#)

LiveKit Meet is an open source video conferencing app built on [LiveKit Components](#), [LiveKit Cloud](#), and Next.js. It's been completely redesigned from the ground up using our new components library.



Tech Stack

- This is a [Next.js](#) project bootstrapped with [create-next-app](#).
- App is built with [@livekit/components-react](#) library.

Demo

Give it a try at <https://meet.livekit.io>.

Dev Setup

Steps to get a local dev setup up and running:

1. Run `pnpm install` to install all dependencies.
2. Copy `.env.example` in the project root and rename it to `.env.local`.
3. Update the missing environment variables in the newly created `.env.local` file.
4. Run `pnpm dev` to start the development server and visit <http://localhost:3000> to see the result.

```
→ livekit git clone git@github.com:livekit-examples/meet.git
→ livekit cd meet
→ meet git:(main) npm install
→ meet git:(main) x cp .env.example .env.development
→ meet git:(main) x vi .env.development
→ meet git:(main) x npm run dev
```

<https://github.com/livekit-examples/meet>

GET STARTED

[Overview](#)
[About LiveKit](#)
[Docs MCP server](#)
[DeepLearning course](#)
[Community resources](#)

UNDERSTANDING LIVEKIT

[Overview](#)
[LiveKit CLI](#) 
 [Overview](#)
 [Setup](#)
 [Project management](#)
 [App templates](#)
[LiveKit Cloud](#)
[Connecting to LiveKit](#)
[Rooms, participants, & tracks](#) 
[Building AI agents](#)

REFERENCE

[Recipes](#)
[Room service API](#)

CLI setup

 Copy page

Install the LiveKit CLI and test your setup using an example frontend application.

Get started

`lk` is LiveKit's suite of CLI utilities. It lets you conveniently access server APIs, create tokens, and generate test traffic from the command line.

Install the CLI

Install the CLI using the following command.

MACOS

LINUX

WINDOWS

FROM SOURCE

Install the LiveKit CLI with [Homebrew](#):

```
brew install livekit-cli
```



For more details, view the `livekit-cli` [GitHub repo](#).

Update the CLI

LiveKit recommends updating the CLI regularly to ensure you have the latest features and bug fixes.

On this page

[Get started](#)[Install the CLI](#)[Update the CLI](#)[Add a project](#)[Generate access token](#)[Test with LiveKit Meet](#)[Simulating another publis...](#)

Is this page helpful?

<https://docs.livekit.io/intro/basics/cli/start/>

GET STARTED

- Overview
- About LiveKit
- Docs MCP server
- DeepLearning course
- Community resources

UNDERSTANDING LIVEKIT

- Overview
- LiveKit CLI
 - Overview
 - Setup**
 - Project management
 - App templates
- LiveKit Cloud
- Connecting to LiveKit
- Rooms, participants, & tracks
- Building AI agents

REFERENCE

- Recipes
- Room service API

A participant creating or joining a LiveKit room needs an access token to do so. For details on token structure and creation, see Token creation. You can generate one using the CLI:

CLOUD LOCALHOST

```
lk token create \  
  --api-key devkey --api-secret secret \  
  --join --room test_room --identity test_user \  
  --valid-for 24h
```



Tip

Make sure you're running LiveKit server locally in dev mode.

Alternatively, you can generate tokens from your project's dashboard.

Test with LiveKit Meet



Tip

If you're testing a LiveKit Cloud instance, you can find your `Project URL` (it starts with `wss://`) in the project settings.

Use a sample app, LiveKit Meet, to preview your new LiveKit instance. Enter the token you previously generated in the "Custom" tab. After you connect, your microphone and camera are streamed in realtime to your new LiveKit instance, and any other participant who connects to the same room.

The full source for this example app is available in GitHub.

Simulating another publisher

One way to test a multi-user session is by generating a second token (ensure `--identity` is unique), opening the LiveKit Meet example app in another browser tab, and connecting to

On this page

- Get started
 - Install the CLI
 - Update the CLI
- Add a project
- Generate access token**
- Test with LiveKit Meet
 - Simulating another publis...

Is this page helpful?



<https://docs.livekit.io/intro/basics/cli/start/>

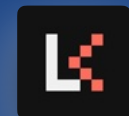
AGENTS & REALTIME VIDEO PROCESSING

Management Services

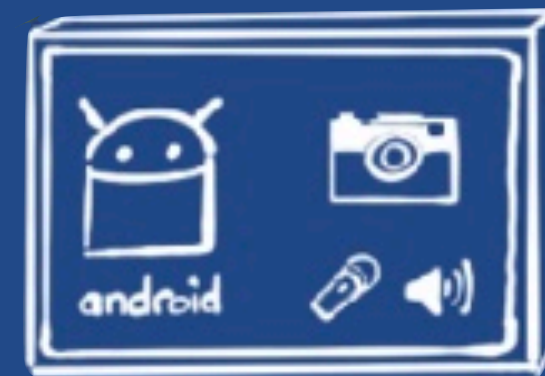
Authentication Service

Room Management Service

Client Terminal Service



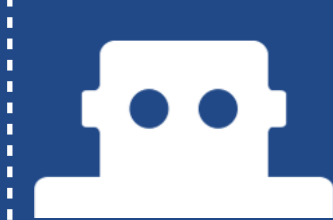
LiveKit Room



Client Terminals



Client



Agent



Support



Agent

Machine Learning Services

PyTorch TensorFlow

Object Detection

Object Classification

Text to Speech

Speech to Text

Generative AI

Natural Language Processing

Logging Services

Video Transcription

Media Stream Recording

Inference History

Audio Transcription



Realtime Media APIs

Image Processing

Encoding and Transcoding

Audio Processing

Filtering

Video Processing

Subtitles and Captioning



PLATFORM ARCHITECTURE DIAGRAM

Shows the components in more details.

AUTHOR: @kanawish
DATE: 24/04/20

To exit full screen, press and hold `esc`

GET STARTED

Introduction

- Voice AI quickstart
- Agent Builder
- Agents playground
- Prompting guide
- Testing & evaluation

MULTIMODALITY

- Overview
- Speech & audio
- Text & transcriptions
- Vision

LOGIC & STRUCTURE

- Overview
- Agent sessions
- Tasks & task groups
- Workflows
- Tool definition & use
- Pipeline nodes & hooks
- Turn detection & interruptions
- Agents & handoffs
- External data & RAG

PREBUILT COMPONENTS

- Prebuilt tasks
- Prebuilt tools

AGENT SERVER

- Overview
- Startup modes
- Server lifecycle
- Agent dispatch
- Job lifecycle
- Server options

MODELS

- Overview
- LLM
- STT
- TTS

Introduction

Realtime framework for voice, video, and physical AI agents.

Overview

The Agents framework lets you add any Python or Node.js program to LiveKit rooms as full realtime participants. Build agents with code using the Python and Node.js SDKs, or use [LiveKit Agent Builder](#) to prototype and deploy agents directly in your browser without writing code. The framework provides tools and abstractions for feeding realtime media and data through an AI pipeline that works with any provider, and publishing realtime results back to the room.

Use LiveKit Cloud to start building agents right away, with managed deployment, built-in observability with transcripts and traces, and LiveKit Inference for running AI models without API keys. You can deploy your agents to [LiveKit Cloud](#) or any [custom environment](#) of your choice.

If you want to get your hands on the code for building an agent right away, follow the [Voice AI quickstart guide](#) or try out [Agent Builder](#) and build your first voice agent in minutes. It takes just a few minutes to build your first voice agent.



Voice AI quickstart
Build and deploy a simple voice assistant with Python or Node.js in less than 10 minutes.



LiveKit Agent Builder
Prototype and deploy voice agents directly in your browser, without writing any code.



Deeplearning.ai course
Learn to build and deploy voice agents with LiveKit in this free course from Deeplearning.ai.



Deploying to LiveKit Cloud
Run your agent on LiveKit Cloud's global infrastructure.



GitHub repository
Python source code and examples for the LiveKit Agents SDK.



SDK reference
Python reference docs for the LiveKit Agents SDK.

On this page

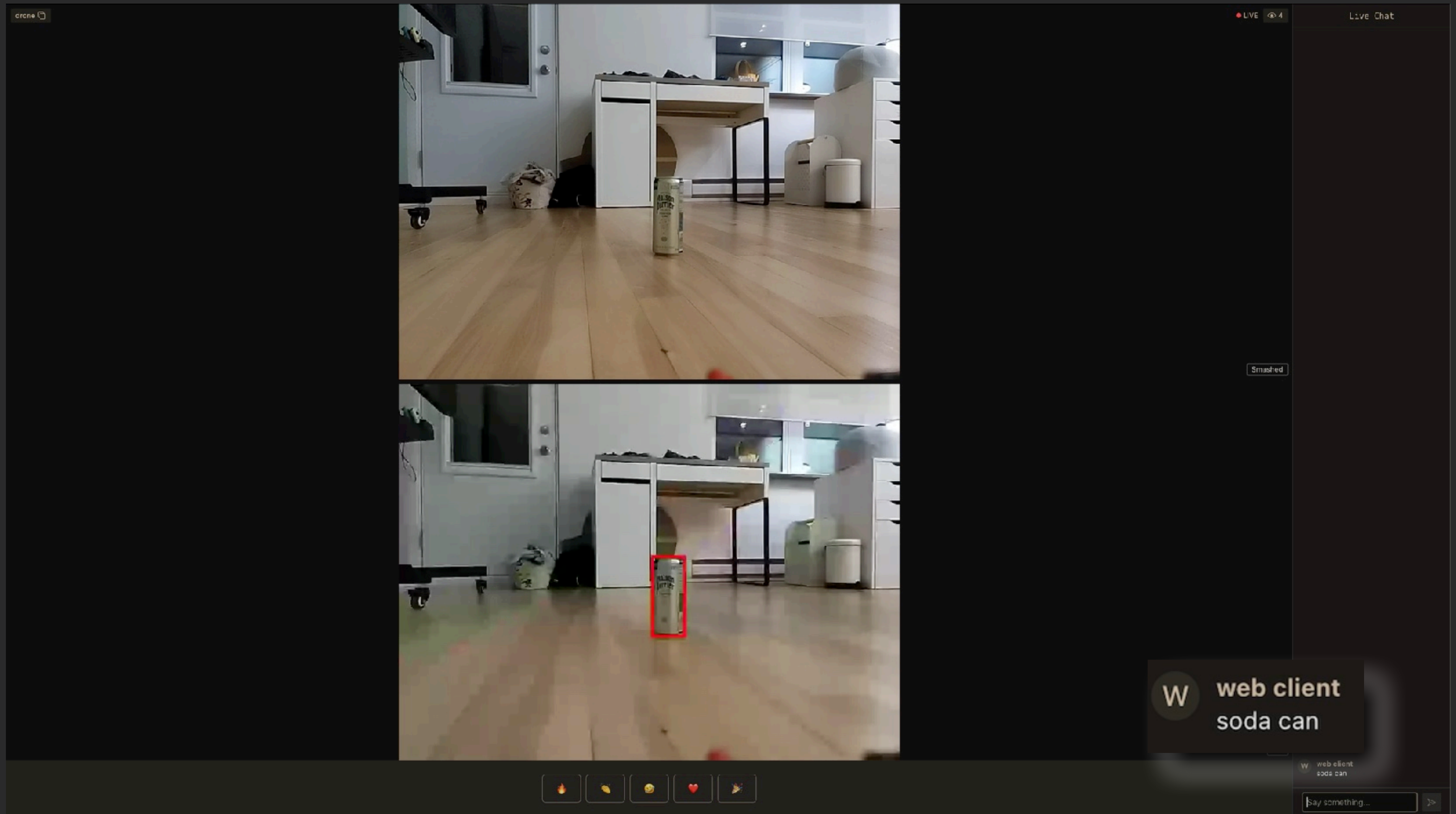
- Overview
 - Use cases
 - Framework overview
 - How agents connect to Liv...
- Key concepts
 - Multimodality
 - Logic & structure
 - Agent server
 - Models
- Getting started

Is this page helpful?





<https://docs.livekit.io/agents/>



Florence 2 image search video - 'Soda can'

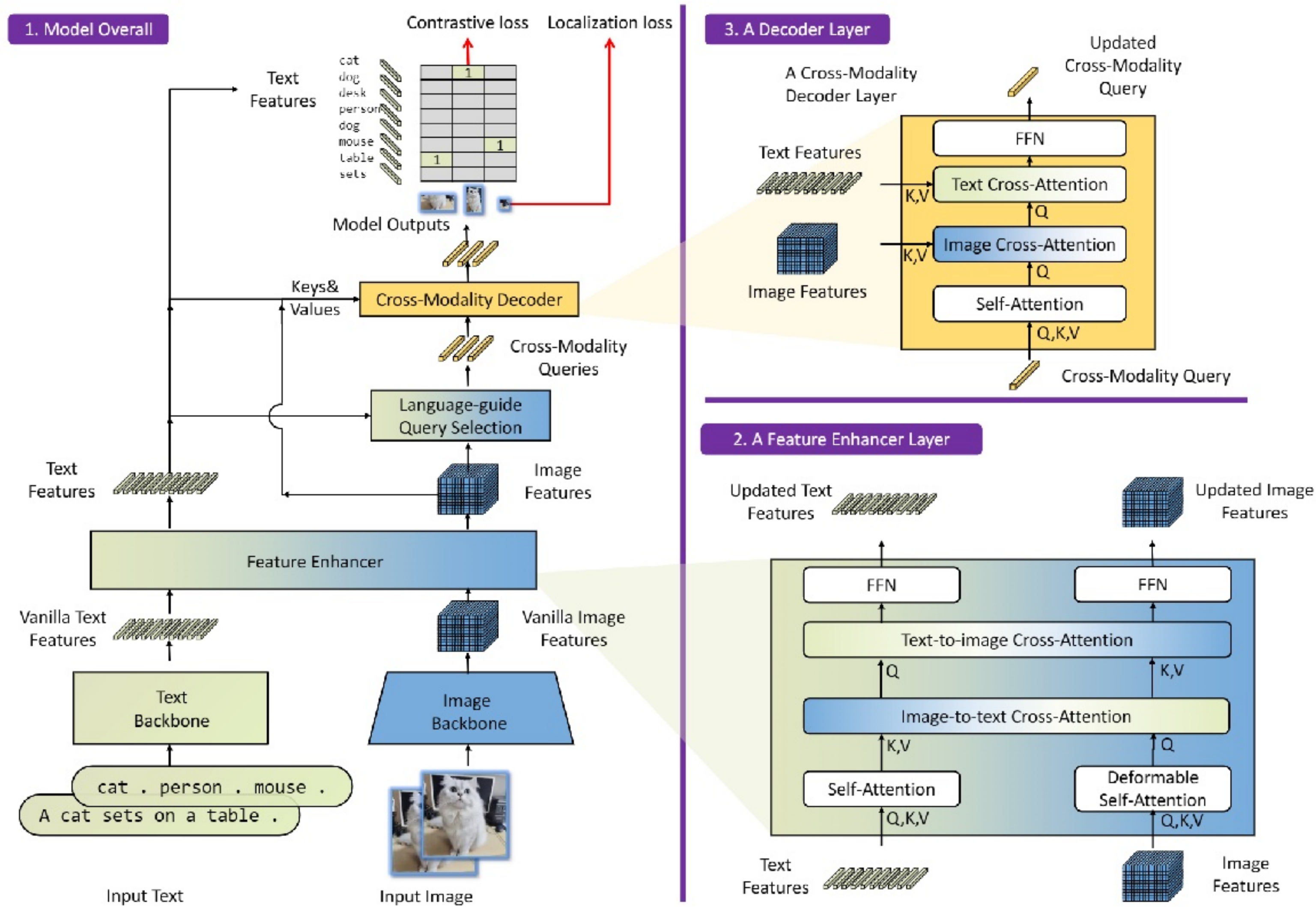


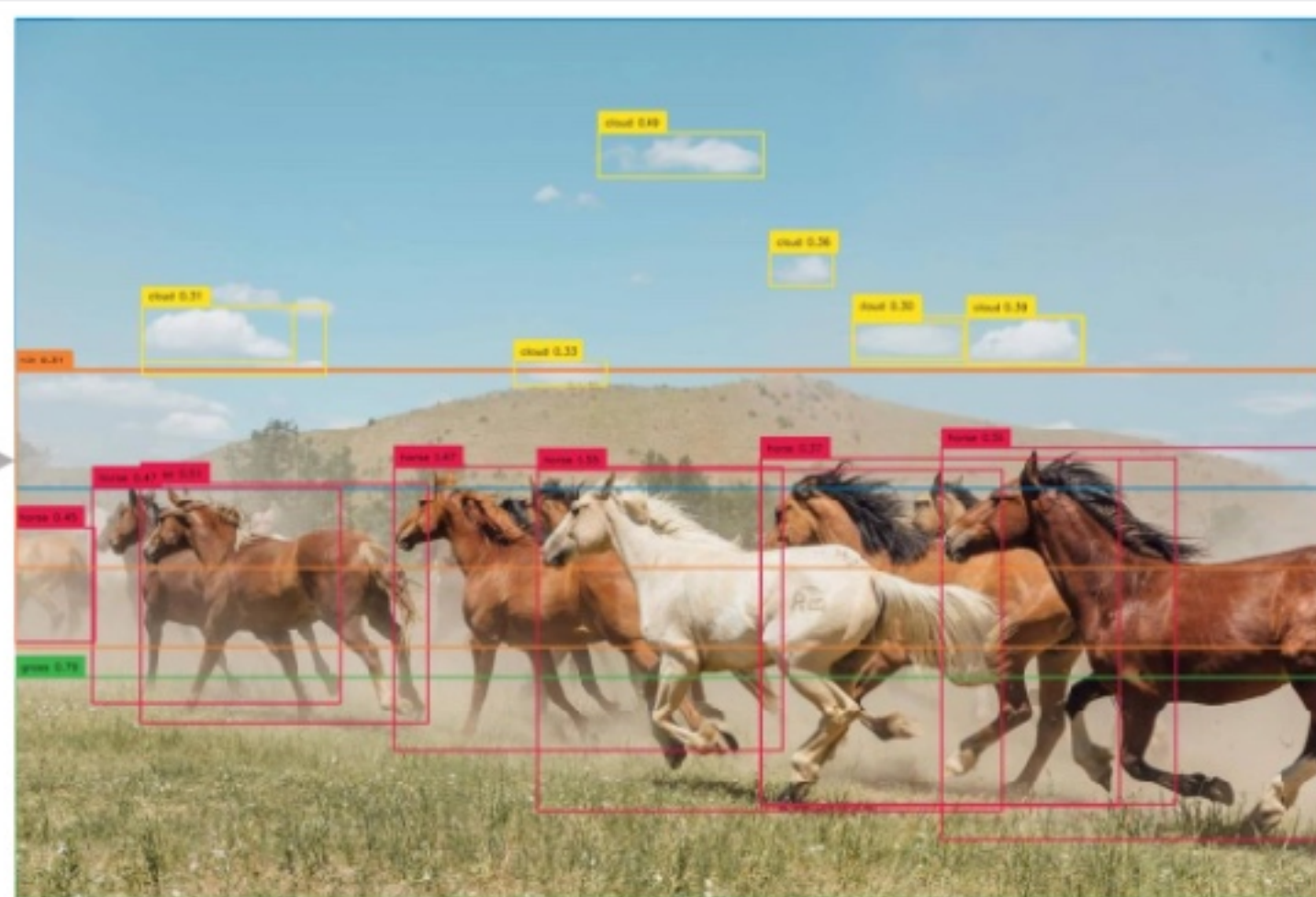
Figure 3. The framework of Grounding DINO. We present the overall framework, a feature enhancer layer, and a decoder layer in block 1, block 2, and block 3, respectively.

Grounded SAM

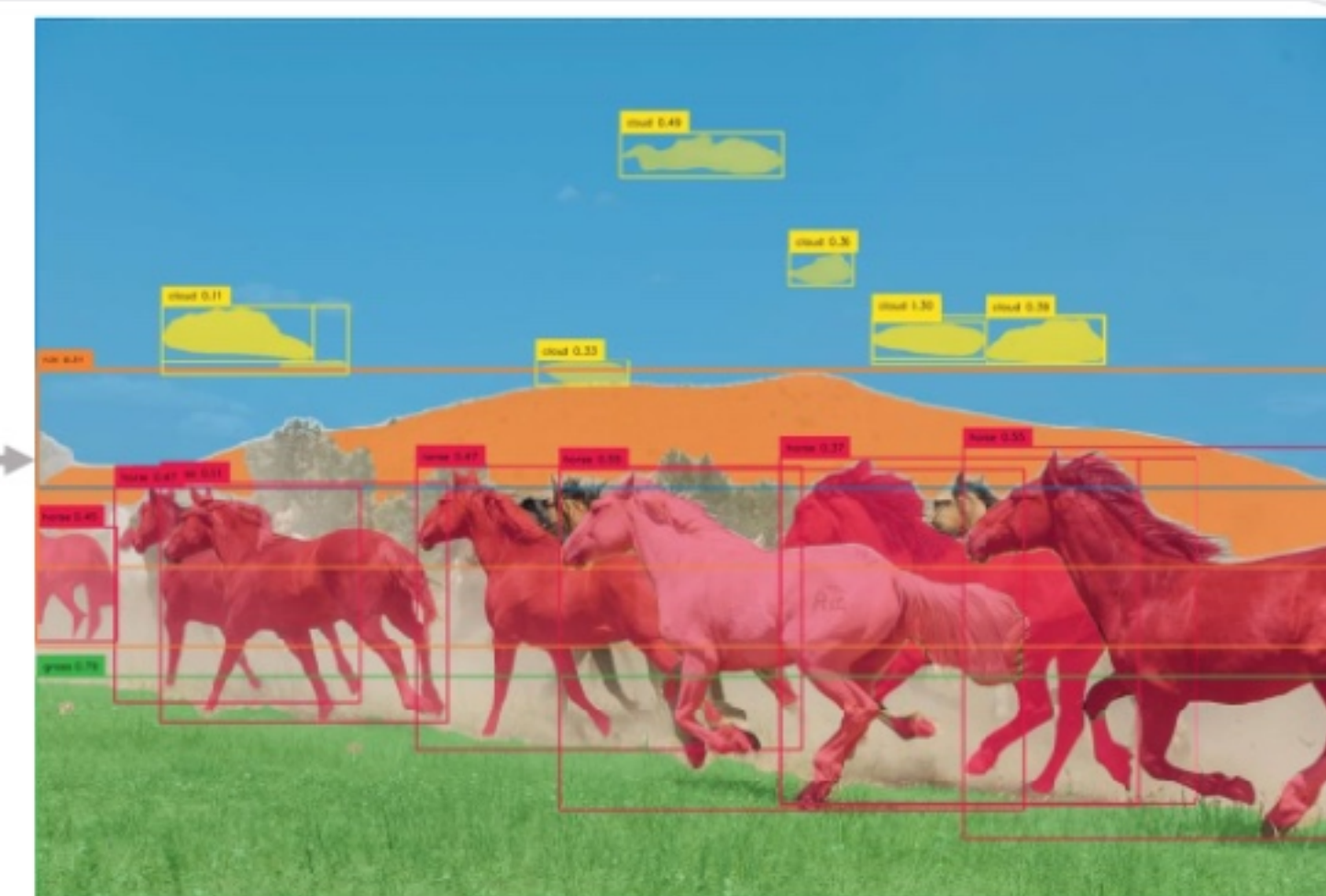
One can combine Grounding DINO with the Segment Anything model for text-based mask generation as introduced in Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks. You can refer to this demo notebook 🌐 for details.



Text Prompt:
“Horse. Clouds. Grasses. Sky. Hill.”

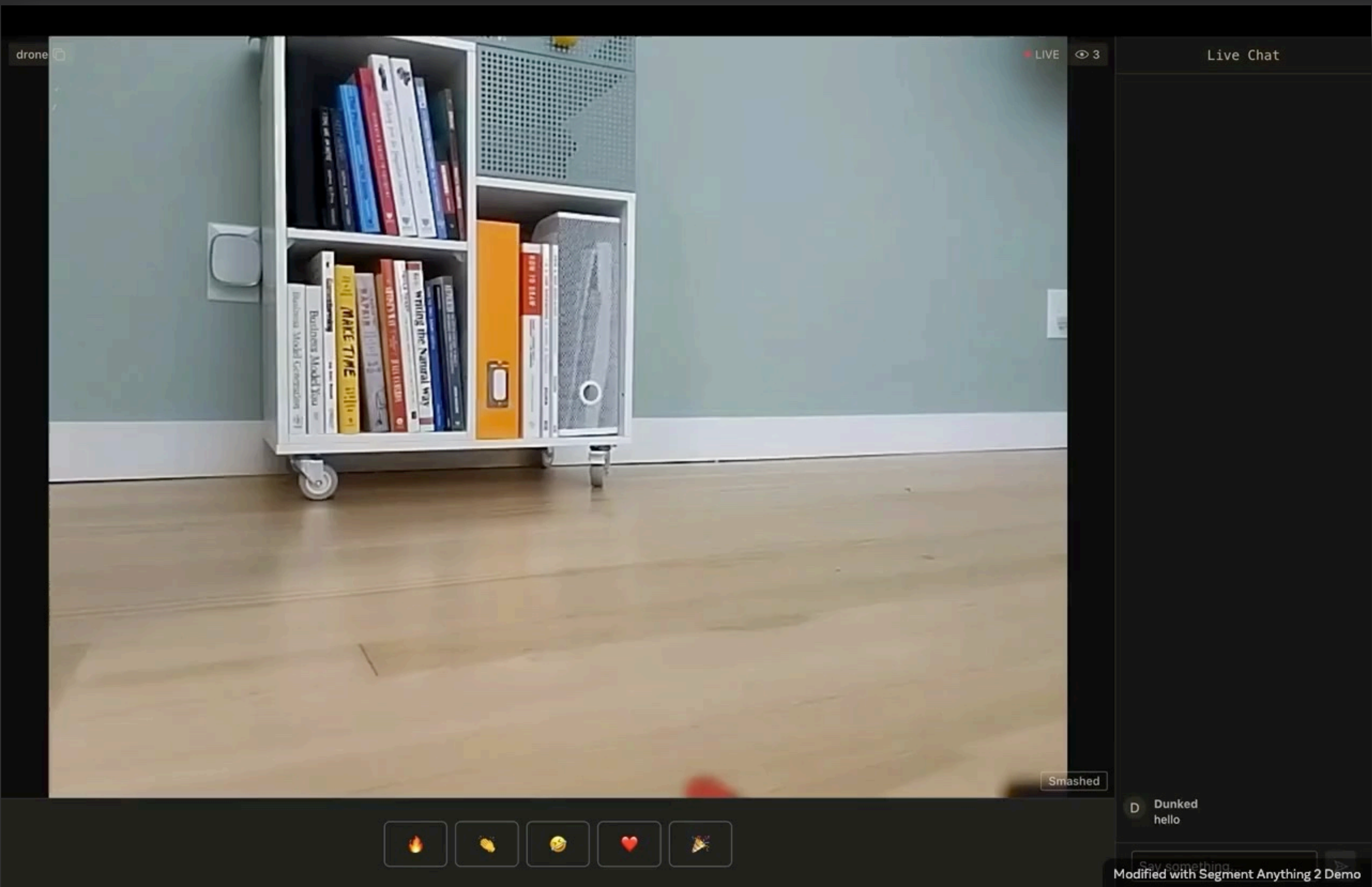


Grounding DINO:
Detect Everything



Grounded-SAM:
Detect and Segment Everything

https://huggingface.co/docs/transformers/en/model_doc/grounding-dino



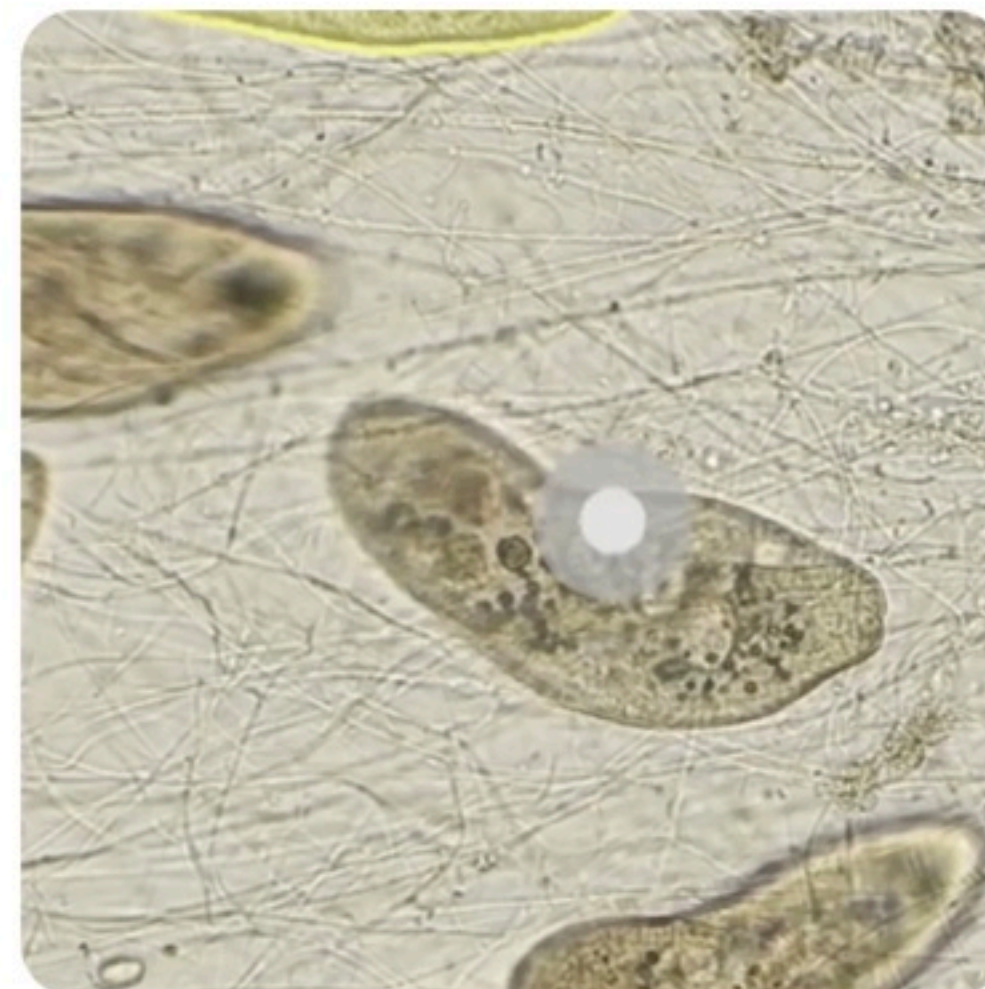
Florence2 prompting SAM2
<https://sam2.metademolab.com/>

→ Read the research paper



Select objects and make adjustments across video frames

Using SAM 2, you can select one or multiple objects in a video frame. Use additional prompts to refine the model predictions.



Robust segmentation, even in unfamiliar videos

SAM 2 is capable of strong zero-shot performance for objects, images and videos not previously seen during model training, enabling use in a wide range of real-world applications.



Real-time interactivity and results

SAM 2 is designed for efficient video processing with streaming inference to enable real-time, interactive applications.

State-of-the-art
performance for object
segmentation



11:59

LTE



Pixel Perfect Depth - a Hugging Face...
huggingface.co



11:33



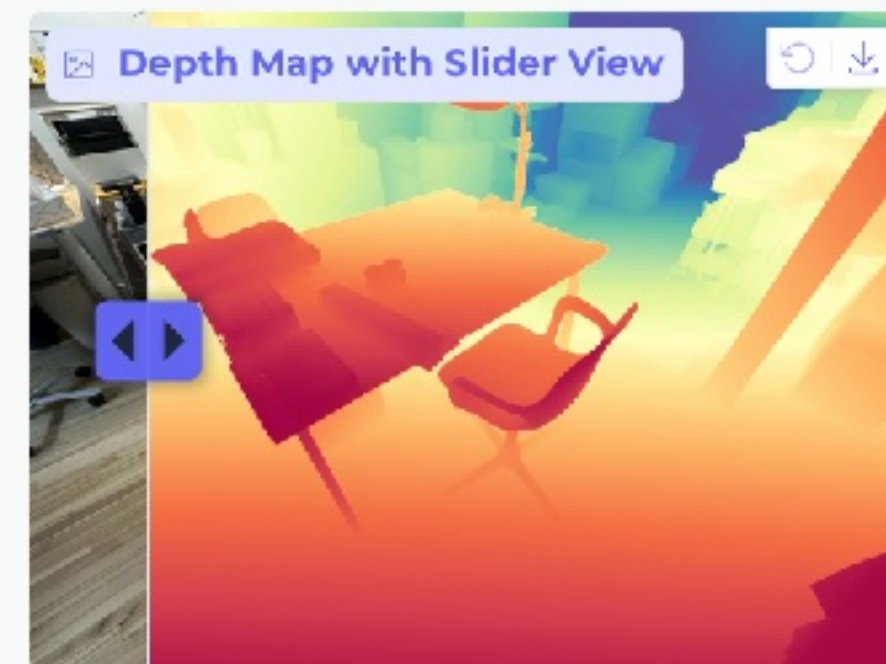
Settings

Predict

3D View

Depth

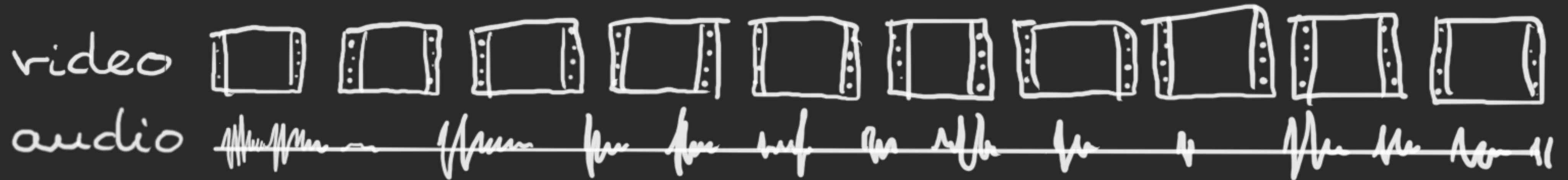
Download



Examples

Room

Tracks



Data Channel (shared)

{ json } ;  ; ...
binary

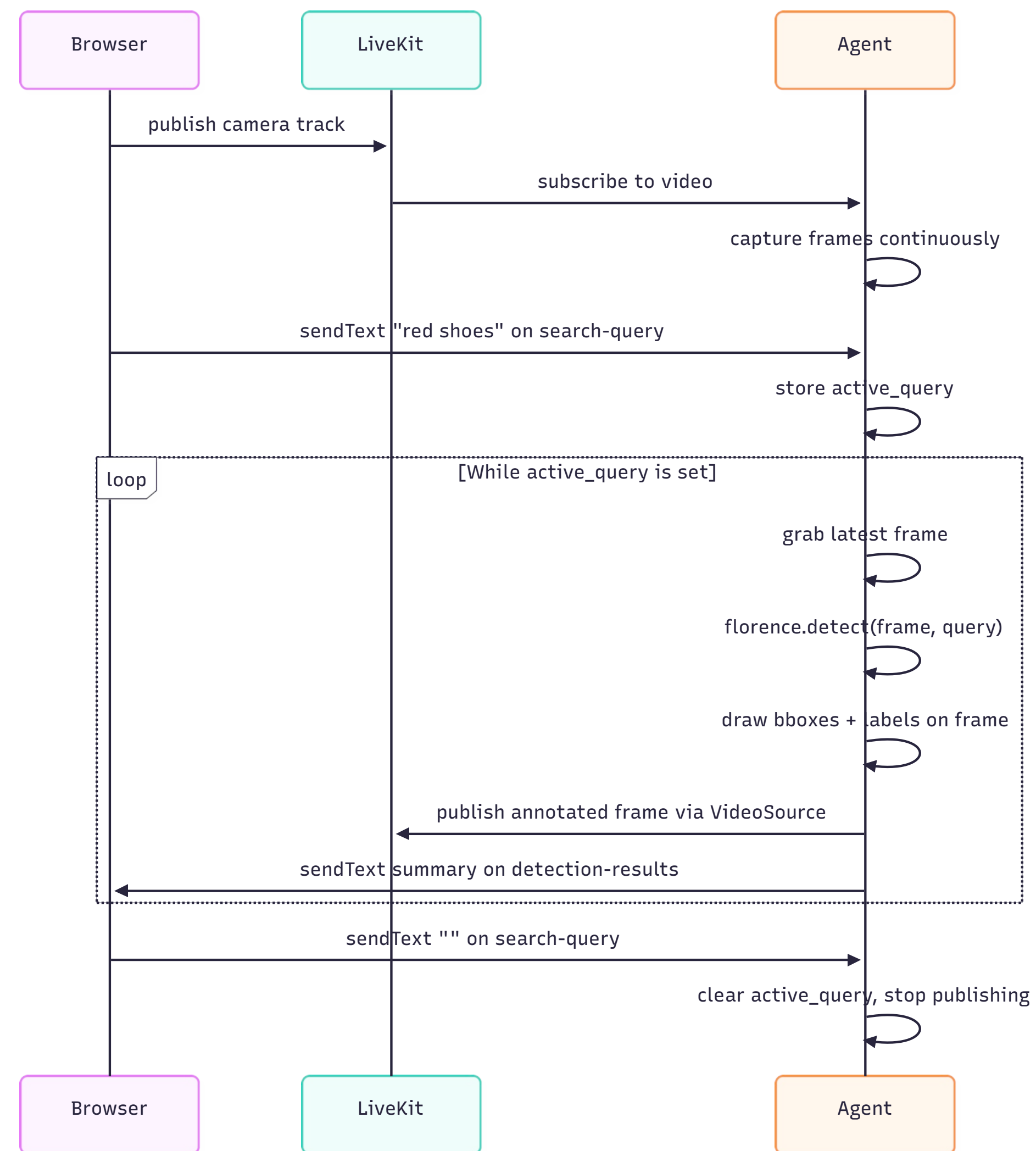
State (shared)

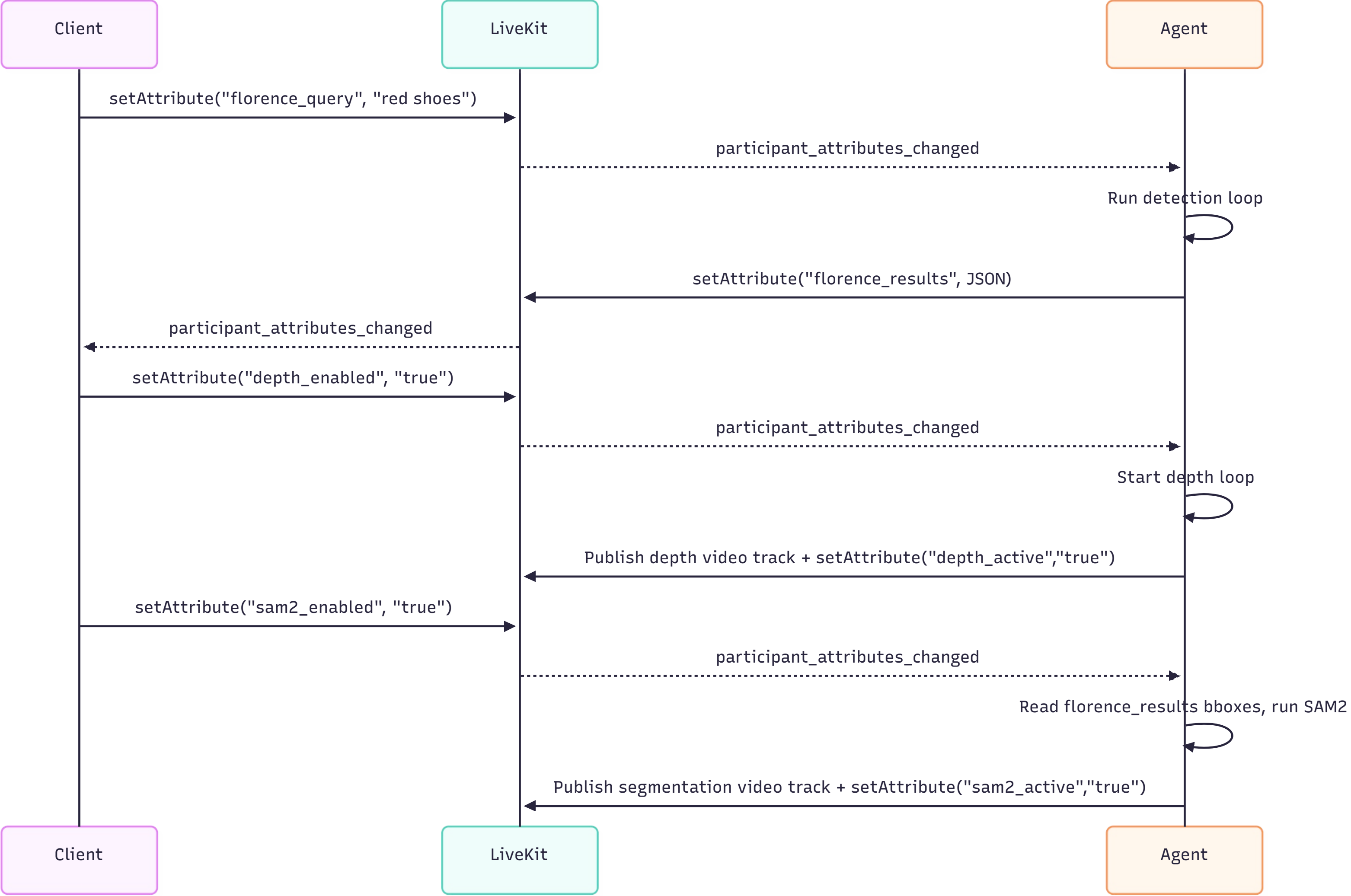


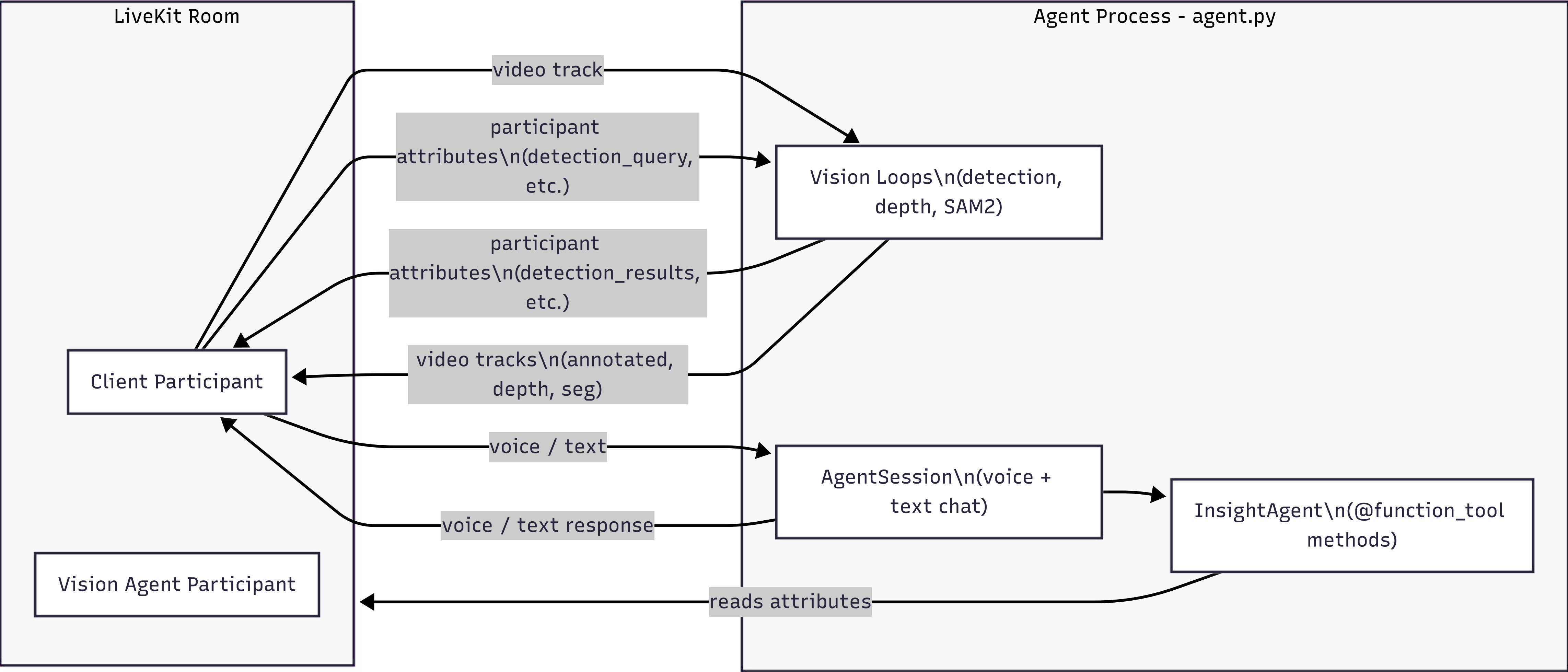
Conceptualizing a Room as Streams & State

**DEMO EFFECT
ACTIVATED**









References

Source Code and Documentation

- <https://docs.livekit.io/home/>
- <https://github.com/livekit/livekit>
- <https://github.com/livekit-examples/livestream>
- <https://github.com/kanawish/control-room>



Official LiveKit Content



Demo Code

